

Conflits Frontend ↔ Backend — Page Réservations

Généré le 2026-05-16. Basé sur l'analyse du swagger backend (/api/reservations/) vs le code frontend (src/pages/reservations.tsx , src/components/reservations/types.ts , src/hooks/use-reservations.ts).

Résumé

Le backend fournit un CRUD basique sur les réservations (client, horaire, taille groupe, salle/table, notes, allergies). Le frontend implémente **beaucoup plus de features** (statuts, pipeline, canal, durée estimée) qui sont **100% côté client uniquement** — rien n'est persisté. Toute action de changement de statut est perdue au rechargement.

Champs manquants côté backend

Champ frontend	Type frontend	Existe côté backend ?	Impact
status	"confirmée" "en_attente" "arrivée" "annulée" "no_show"	NON	Critique — Le statut est hardcodé à "confirmée" au mapping. Les changements de statut sont UI-only (localStorage-like), perdus au refresh.
canal	"site" "telephone" "thefork" "walk_in"	NON	Moyen — Hardcodé à "telephone". Aucune traçabilité de l'origine de la résa.
estimatedDurationMinutes	number	NON	Moyen — Utilisé par le gantt pour calculer la largeur des blocs. Actuellement fallback à une valeur par défaut.
pipeline / currentStageId	{ currentStageId: string, history: [...] }	NON	Faible (feature avancée) — Le pipeline stepper dans le détail est purement décoratif sans persistance.
clientEmail	string	NON	Faible — Optionnel, jamais envoyé au backend.
createdAt	string	NON	Faible — Le frontend le set à "". Pas de date de création exposée par l'API.
service (midi/soir)	"midi" "soir"	Filtre uniquement — Le backend accepte ?service=midi en query param mais ne le stocke pas comme champ. Le frontend le déduit de l'heure (< 16h = midi).	Correct (pas un conflit).

Actions non fonctionnelles (UI-only, non persistées)

Action UI	Ce qui se passe réellement
-----------	----------------------------

Changer le statut (confirmer, marquer arrivée, annuler, no-show)	Mise à jour local overrides en mémoire uniquement. Perdu au refresh.
Avancer le pipeline	Purement décoratif. Aucune mutation API.
Filtrer par statut	Fonctionne sur les données locales, mais tous les statuts sont "confirmée" depuis l'API.
Voir le canal	Toujours "Téléphone" (default).
Gantt : durée des blocs	Calculée via estimatedDurationMinutes qui n'existe pas — fallback au frontend.

Actions fonctionnelles (correctement intégrées)

Action UI	Endpoint backend	Fonctionne ?
Créer une réservation	POST /api/reservations/	✓
Modifier notes (note_serveur)	PATCH /api/reservations/{id}/	✓ (optimistic update + revert on error)
Modifier horaire (reschedule)	PATCH /api/reservations/{id}/ (datetime)	✓
Modifier table assignée	PATCH /api/reservations/{id}/ (table_id)	✓
Supprimer réservation	DELETE /api/reservations/{id}/	✓ (hook existe, pas encore câblé dans l'UI)
Lister par date + restaurant	GET /api/reservations/?date=X&restaurant_id=Y	✓
Lister les salles	GET /api/salles/?restaurant_id=Y	✓
Lister les tables	GET /api/tables/?salle_id=Z	✓

Ce qu'il faudrait ajouter côté backend

Priorité 1 — CRITIQUE (feature non fonctionnelle sans ça)

1. Champ `status` sur le modèle Reservation

```
# models.py
class Reservation(models.Model):
    STATUS_CHOICES = [
        ("en_attente", "En attente"),
        ("confirmée", "Confirmée"),
        ("arrivée", "Arrivée"),
        ("annulée", "Annulée"),
        ("no_show", "No-show"),
    ]
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default="en_attente",
    )
```

- Exposer dans le serializer (read + write)
- Ajouter au `PatchedReservationRequest` (permettre PATCH du statut seul)
- Ajouter un filtre `?status=confirmée` sur la liste

Impact frontend : Supprimer le `hardcode` `status: "confirmée"` dans `mapApiReservation`, mapper directement `api.status`. Le `handleStatusChange` devient un vrai `PATCH` au lieu d'un `setLocalOverrides`.

2. Champ `canal` (source de la réservation)

```
CANAL_CHOICES = [
    ("site", "Site web"),
    ("telephone", "Téléphone"),
    ("thefork", "TheFork"),
    ("walk_in", "Walk-in"),
]
canal = models.CharField(
    max_length=20,
    choices=CANAL_CHOICES,
    default="telephone",
)
```

- Exposer en lecture dans le serializer
 - Writable à la création (pas modifiable après ?)
 - Permet d'afficher correctement la provenance dans le tableau + détail
-

Priorité 2 — IMPORTANT (UX dégradée sans ça)

3. Champ `estimated_duration_minutes`

```
estimated_duration_minutes = models.PositiveIntegerField(
    default=90,
    help_text="Durée estimée du repas en minutes",
)
```

- Valeur par défaut intelligente (90min midi, 120min soir ?)
 - Utilisé par le gantt pour la largeur des blocs et les alertes de chevauchement
 - Permettre override par le gérant
-

4. Champ `created_at` (auto)

```
created_at = models.DateTimeField(auto_now_add=True)
```

- Read-only dans le serializer
 - Permet un tri par date de création dans le tableau
-

Priorité 3 — NICE TO HAVE (feature avancée)

5. Pipeline / workflow de service

C'est un système plus complexe. Options :

- **Option A — Simple :** Un champ `service_stage` (enum: `attente_client`, `installe`, `commande_prise`, `service_en_cours`, `dessert`, `addition`, `parti`) sur le modèle `Reservation`.
- **Option B — Flexible :** Un modèle séparé `ReservationPipelineStep` avec FK vers `Reservation` + timestamps.

Le frontend a déjà toute la UI (`PipelineStepper` , `PipelineTemplate` , `PipelineStageDefinition`). Il manque juste la persistance.

6. Champ `client_email`

```
client_email = models.EmailField(blank=True, null=True)
```

Optionnel, utile pour les confirmations par email futures.

Résumé des actions

#	Quoi	Côté	Priorité	Effort estimé
1	Ajouter status au modèle + serializer + filtre	Backend	P1	~1h
2	Ajouter canal au modèle + serializer	Backend	P1	~30min
3	Ajouter <code>estimated_duration_minutes</code>	Backend	P2	~30min
4	Ajouter <code>created_at</code>	Backend	P2	~15min
5	Pipeline / workflow	Backend	P3	~4h (option B)
6	Ajouter <code>client_email</code>	Backend	P3	~15min
7	Supprimer <code>localOVERRIDES</code> status, mapper <code>api.status</code>	Frontend	P1 (après #1)	~30min
8	Mapper <code>api.canal</code> au lieu du hardcode	Frontend	P1 (après #2)	~10min
9	Mapper <code>api.estimatedDurationMinutes</code>	Frontend	P2 (après #3)	~10min

Migration suggérée

```
# Backend – après ajout des champs
python manage.py makemigrations reservations
python manage.py migrate

# Migration data : toutes les résas existantes → status="confirmée", canal="telephone"
# (ce sont les defaults hardcodés actuellement côté frontend)
```

Après déploiement backend :

1. Régénérer les types Orval : `pnpm gen:api`
2. Supprimer les hardcodes dans `mapApiReservation`
3. Remplacer `setLocalOVERRIDES` par un vrai `PATCH` pour le statut
4. Supprimer le commentaire `// TODO: persist when backend adds status field`